Verificación de Lossless Join

Aseguramiento de calidad del diseño lógico

1. Método

Para cada par de relaciones unidas por una FK se aplica la prueba binaria:

> **$(R_1 \cap R_2) \rightarrow R_1$ o $(R_1 \cap R_2) \rightarrow R_2$**
> El atributo común debe determinar completamente al menos una de las dos relaciones.

2. Dependencias funcionales

Cada PK determina todos los atributos de su tabla, incluidas sus FK:

Dependencia funcional	Tabla donde se verifica
id_employee → name, email, phone, salary, zone	EMPLOYEES
nit → name, phone, email	SUPPLIERS
id_purchase → purchase_date, total_price, nit	PURCHASESUPPLIERS
id_purchase_detail → quantity, price, purchase_unit, id_purchase, id_ingredient	PURCHASEDETAILS
id_ingredient → name, ingredient_unit, minimum_stock, current_stock, id_detail	INGREDIENTS
id_detail → quantity, restaurant_price, plate_unit, id_plate	PLATE_DETAILS
id_plate → name, price, description	PLATES
id_order → order_date, hour, total_price, status, id_employee	ORDERS
id_order_detail → plate_quantity, id_order, id_plate	ORDERDETAILS
id_movement → movement_date, type, id_ingredient, id_employee, id_purchase, id_order	MOVEMENTS

3. Datos de ejemplo

EMPLOYEES

id_employee	name	email	phone	salary	zone
E1	Ana	ana@rest.com	3001112233	1.800.000	Terraza
E2	Luis	luis@rest.com	3004445566	1.800.000	Salón

id_employee	name	email	phone	salary	zone
E3	Marta	marta@rest.com	3007778899	3.500.000	NULL

SUPPLIERS

nit	name	phone	email
121211	Proveedor X	12345	p@gmail.com
333344	Proveedor Y	67890	y@gmail.com

PURCHASESUPPLIERS

id_purchase	purchase_date	total_price	nit
3131	2026-09-01	32.000	121211
3232	2026-09-15	55.000	121211
4242	2026-09-08	120.000	333344

PURCHASEDETAILS

id_purchase_detail	quantity	price	purchase_unit	id_purchase	id_ingredient
PD1	10	20.000	kilogramos	3131	I1
PD2	6	12.000	unidades	3131	I2
PD3	8	105.000	kilogramos	4242	I3
PD4	5	15.000	kilogramos	4242	I1
PD5	20	55.000	unidades	3232	I2

INGREDIENTS

id_ingredient	name	ingredient_unit	minimum_stock	current_stock	id_detail
I1	Arroz	kilogramos	20	45	D1
I2	Maduro	unidades	30	60	D2
I3	Pollo	kilogramos	15	25	D3

PLATE_DETAILS

id_detail	quantity	restaurant_price	plate_unit	id_plate
D1	150	1.200	gramos	P1
D2	1	2.000	unidades	P1
D3	200	6.000	gramos	P2

PLATES

id_plate	name	price	description
P1	Bandeja paisa	25.000	Plato típico antioqueño
P2	Arroz con pollo	18.000	Arroz con pollo desmechado

ORDERS

id_order	order_date	hour	total_price	status	id_employee
O1	2026-10-01	12:30	43.000	Entregado	E1
O2	2026-10-01	13:15	54.000	En preparación	E1
O3	2026-10-02	19:00	50.000	Entregado	E2

ORDERDETAILS

id_order_detail	plate_quantity	id_order	id_plate
OD1	1	O1	P1
OD2	1	O1	P2
OD3	3	O2	P2
OD4	2	O3	P1

MOVEMENTS

id_movement	movement_date	type	id_ingredient	id_employee	id_purchase	id_order
M1	2026-09-01	Entrada	I1	E3	3131	NULL
M2	2026-09-01	Entrada	I2	E3	3131	NULL
M3	2026-09-08	Entrada	I3	E3	4242	NULL
M4	2026-09-08	Entrada	I1	E3	4242	NULL
M5	2026-09-15	Entrada	I2	E3	3232	NULL
M6	2026-10-01	Salida	I1	E1	NULL	O1
M7	2026-10-01	Salida	I2	E1	NULL	O1
M8	2026-10-01	Salida	I3	E1	NULL	O1
M9	2026-10-01	Salida	I3	E1	NULL	O2
M10	2026-10-02	Salida	I1	E2	NULL	O3
M11	2026-10-02	Salida	I2	E2	NULL	O3

STATUS

status
Entregado
En preparación
Cancelado

TYPE

type
Entrada
Salida

4. Verificación de cada JOIN

1. PURCHASESUPPLIERS ⋈ SUPPLIERS

COMPRAS_PROVEEDOR = PURCHASESUPPLIERS ⋈ SUPPLIERS → hecho que reconstruye: compras realizadas a un proveedor.

Resultado del JOIN

id_purchase	purchase_date	total_price	nit	name	phone	email
3131	2026-09-01	32.000	121211	Proveedor X	12345	p@gmail.com
3232	2026-09-15	55.000	121211	Proveedor X	12345	p@gmail.com
4242	2026-09-08	120.000	333344	Proveedor Y	67890	y@gmail.com

Prueba de lossless join (diapositiva 20)

PURCHASESUPPLIERS ∩ SUPPLIERS = {nit}

DF: nit → name, phone, email (nit es la PK de SUPPLIERS)

$\{nit\}^+ \supseteq$ SUPPLIERS, por tanto (PURCHASESUPPLIERS ∩ SUPPLIERS) → SUPPLIERS.

∴ El JOIN es lossless.

Verificación con el ejemplo: PURCHASESUPPLIERS tiene 3 filas y el resultado tiene 3; cada fila de PURCHASESUPPLIERS se une con exactamente una fila de SUPPLIERS, sin tuplas espurias.

2. PURCHASEDETAILS ⋈ PURCHASESUPPLIERS

DETALLES_COMPRA = PURCHASEDETAILS ⋈ PURCHASESUPPLIERS → hecho que reconstruye: detalles (ítems) de una compra.

Resultado del JOIN

id_purchase_detail	quantity	price	purchase_unit	id_purchase	id_ingredient	purchase_date	total_price	nit
PD1	10	20.000	kilogramos	3131	I1	2026-09-01	32.000	121211
PD2	6	12.000	unidades	3131	I2	2026-09-01	32.000	121211
PD3	8	105.000	kilogramos	4242	I3	2026-09-08	120.000	333344
PD4	5	15.000	kilogramos	4242	I1	2026-09-08	120.000	333344
PD5	20	55.000	unidades	3232	I2	2026-09-15	55.000	121211

Prueba de lossless join

PURCHASEDETAILS ∩ PURCHASESUPPLIERS = {id_purchase}

DF: id_purchase → purchase_date, total_price, nit (id_purchase es la PK de PURCHASESUPPLIERS)

$\{id_purchase\}^+ \supseteq$ PURCHASESUPPLIERS, por tanto (PURCHASEDETAILS ∩ PURCHASESUPPLIERS) → PURCHASESUPPLIERS.

∴ El JOIN es lossless.

Verificación con el ejemplo: PURCHASEDETAILS tiene 5 filas y el resultado tiene 5; cada fila de PURCHASEDETAILS se une con exactamente una fila de PURCHASESUPPLIERS, sin tuplas espurias.

3. PURCHASEDETAILS ⋈ INGREDIENTS

INGREDIENTES_COMPRADOS = PURCHASEDETAILS ⋈ INGREDIENTS → hecho que reconstruye: qué ingrediente se compró en cada detalle.

Resultado del JOIN

id_purchase_detail	quantity	price	purchase_unit	id_purchase	id_ingredient	name	ingredient_unit	minimum_stock	current_stock	id_detail
PD1	10	20.000	kilogramos	3131	I1	Arroz	kilogramos	20	45	D1
PD2	6	12.000	unidades	3131	I2	Maduro	unidades	30	60	D2
PD3	8	105.000	kilogramos	4242	I3	Pollo	kilogramos	15	25	D3
PD4	5	15.000	kilogramos	4242	I1	Arroz	kilogramos	20	45	D1
PD5	20	55.000	unidades	3232	I2	Maduro	unidades	30	60	D2

Prueba de lossless join

PURCHASEDETAILS ∩ INGREDIENTS = {id_ingredient}

DF: id_ingredient → name, ingredient_unit, minimum_stock, current_stock, id_detail (id_ingredient es la PK de INGREDIENTS)

$\{id_ingredient\}^+ \supseteq$ INGREDIENTS, por tanto (PURCHASEDETAILS ∩ INGREDIENTS) → INGREDIENTS.

∴ El JOIN es lossless.

Verificación con el ejemplo: PURCHASEDETAILS tiene 5 filas y el resultado tiene 5; cada fila de PURCHASEDETAILS se une con exactamente una fila de INGREDIENTS, sin tuplas espurias.

4. MOVEMENTS ⋈ PURCHASESUPPLIERS

MOVIMIENTOS_COMPRA = MOVEMENTS ⋈ PURCHASESUPPLIERS → hecho que reconstruye: movimientos de inventario (entradas) generados por una compra.

Resultado del JOIN

id_movement	movement_date	type	id_ingredient	id_employee	id_purchase	id_order	purchase_date	total_price	nit
M1	2026-09-01	Entrada	I1	E3	3131	NULL	2026-09-01	32.000	121211
M2	2026-09-01	Entrada	I2	E3	3131	NULL	2026-09-01	32.000	121211
M3	2026-09-08	Entrada	I3	E3	4242	NULL	2026-09-08	120.000	333344
M4	2026-09-08	Entrada	I1	E3	4242	NULL	2026-09-08	120.000	333344
M5	2026-09-15	Entrada	I2	E3	3232	NULL	2026-09-15	55.000	121211

Prueba de lossless joi

MOVEMENTS ∩ PURCHASESUPPLIERS = {id_purchase}

DF: id_purchase → purchase_date, total_price, nit (id_purchase es la PK de PURCHASESUPPLIERS)

$\{id_purchase\}^+ \supseteq$ PURCHASESUPPLIERS, por tanto (MOVEMENTS ∩ PURCHASESUPPLIERS) → PURCHASESUPPLIERS.

∴ El JOIN es lossless.

Verificación con el ejemplo: los 5 movimientos de tipo Entrada se unen cada uno con exactamente una compra, y el resultado tiene 5 filas. Los movimientos de Salida tienen id_purchase = NULL, por lo que no participan: no pertenecen al hecho "movimiento generado por una compra", así que no es pérdida de información. Cada compra genera un movimiento por cada ingrediente comprado (3131 → M1 y M2).

5. MOVEMENTS ⋈ ORDERS

MOVIMIENTOS_PEDIDO = MOVEMENTS ⋈ ORDERS → hecho que reconstruye: movimientos de inventario (salidas) generados por un pedido.

Resultado del JOIN

id_movement	movement_date	type	id_ingredient	id_employee	id_purchase	id_order	order_date	hour	total_price	status	ORDERS.id_employee
M6	2026-10-01	Salida	I1	E1	NULL	O1	2026-10-01	12:30	43.000	Entregado	E1
M7	2026-10-01	Salida	I2	E1	NULL	O1	2026-10-01	12:30	43.000	Entregado	E1
M8	2026-10-01	Salida	I3	E1	NULL	O1	2026-10-01	12:30	43.000	Entregado	E1
M9	2026-10-01	Salida	I3	E1	NULL	O2	2026-10-01	13:15	54.000	En preparación	E1
M10	2026-10-02	Salida	I1	E2	NULL	O3	2026-10-02	19:00	50.000	Entregado	E2
M11	2026-10-02	Salida	I2	E2	NULL	O3	2026-10-02	19:00	50.000	Entregado	E2

Prueba de lossless join

MOVEMENTS ∩ ORDERS = {id_order}

DF: id_order → order_date, hour, total_price, status, id_employee (id_order es la PK de ORDERS)

$\{id_order\}^+ \supseteq$ ORDERS, por tanto (MOVEMENTS ∩ ORDERS) → ORDERS.

∴ El JOIN es lossless.

Verificación con el ejemplo: los 6 movimientos de tipo Salida (M6–M11) se unen cada uno con exactamente un pedido, y el resultado tiene 6 filas. Los movimientos de Entrada tienen id_order = NULL y no participan.

6. MOVEMENTS ⋈ EMPLOYEES

MOVIMIENTOS_EMPLEADO = MOVEMENTS ⋈ EMPLOYEES → hecho que reconstruye: movimientos registrados por un empleado.

Resultado del JOIN

id_movement	movement_date	type	id_ingredient	id_employee	id_purchase	id_order	name	email	phone	salary	zone
M1	2026-09-01	Entrada	I1	E3	3131	NULL	Marta	marta@rest.com	3007778899	3.500.000	NULL
M2	2026-09-01	Entrada	I2	E3	3131	NULL	Marta	marta@rest.com	3007778899	3.500.000	NULL
M3	2026-09-08	Entrada	I3	E3	4242	NULL	Marta	marta@rest.com	3007778899	3.500.000	NULL
M4	2026-09-08	Entrada	I1	E3	4242	NULL	Marta	marta@rest.com	3007778899	3.500.000	NULL
M5	2026-09-15	Entrada	I2	E3	3232	NULL	Marta	marta@rest.com	3007778899	3.500.000	NULL
M6	2026-10-01	Salida	I1	E1	NULL	O1	Ana	ana@rest.com	3001112233	1.800.000	Terraza
M7	2026-10-01	Salida	I2	E1	NULL	O1	Ana	ana@rest.com	3001112233	1.800.000	Terraza
M8	2026-10-01	Salida	I3	E1	NULL	O1	Ana	ana@rest.com	3001112233	1.800.000	Terraza
M9	2026-10-01	Salida	I3	E1	NULL	O2	Ana	ana@rest.com	3001112233	1.800.000	Terraza
M10	2026-10-02	Salida	I1	E2	NULL	O3	Luis	luis@rest.com	3004445566	1.800.000	Salón

id_movement	movement_date	type	id_ingredient	id_employee	id_purchase	id_order	name	email	phone	salary	zone
M11	2026-10-02	Salida	I2	E2	NULL	O3	Luis	luis@rest.com	3004445566	1.800.000	Salón

Prueba de lossless join

MOVEMENTS ∩ EMPLOYEES = {id_employee}

DF: id_employee → name, email, phone, salary, zone (id_employee es la PK de EMPLOYEES)

$\{id_employee\}^+ \supseteq$ EMPLOYEES, por tanto (MOVEMENTS ∩ EMPLOYEES) → EMPLOYEES.

∴ El JOIN es lossless.

Verificación con el ejemplo: MOVEMENTS tiene 11 filas y el resultado tiene 11; cada fila de MOVEMENTS se une con exactamente una fila de EMPLOYEES, sin tuplas espurias.

Marta (E3) es administradora, por eso su zone es NULL. El JOIN se hace por id_employee, no por zone, así que el nulo no afecta la reconstrucción.

7. MOVEMENTS ⋈ INGREDIENTS

MOVIMIENTOS_INGREDIENTE = MOVEMENTS ⋈ INGREDIENTS → hecho que reconstruye: movimientos de un ingrediente.

Resultado del JOIN

id_movement	movement_date	type	id_ingredient	id_employee	id_purchase	id_order	name	ingredient_unit	minimum_stock	current_stock	id_detail
M1	2026-09-01	Entrada	I1	E3	3131	NULL	Arroz	kilogramos	20	45	D1
M2	2026-09-01	Entrada	I2	E3	3131	NULL	Maduro	unidades	30	60	D2
M3	2026-09-08	Entrada	I3	E3	4242	NULL	Pollo	kilogramos	15	25	D3
M4	2026-09-08	Entrada	I1	E3	4242	NULL	Arroz	kilogramos	20	45	D1
M5	2026-09-15	Entrada	I2	E3	3232	NULL	Maduro	unidades	30	60	D2
M6	2026-10-01	Salida	I1	E1	NULL	O1	Arroz	kilogramos	20	45	D1
M7	2026-10-01	Salida	I2	E1	NULL	O1	Maduro	unidades	30	60	D2
M8	2026-10-01	Salida	I3	E1	NULL	O1	Pollo	kilogramos	15	25	D3
M9	2026-10-01	Salida	I3	E1	NULL	O2	Pollo	kilogramos	15	25	D3
M10	2026-10-02	Salida	I1	E2	NULL	O3	Arroz	kilogramos	20	45	D1
M11	2026-10-02	Salida	I2	E2	NULL	O3	Maduro	unidades	30	60	D2

Prueba de lossless join

MOVEMENTS ∩ INGREDIENTS = {id_ingredient}

DF: id_ingredient → name, ingredient_unit, minimum_stock, current_stock, id_detail (id_ingredient es la PK de INGREDIENTS)

{id_ingredient}⁺ ⊇ INGREDIENTS, por tanto (MOVEMENTS ∩ INGREDIENTS) → INGREDIENTS.

∴ El JOIN es lossless.

Verificación con el ejemplo: MOVEMENTS tiene 11 filas y el resultado tiene 11; cada fila de MOVEMENTS se une con exactamente una fila de INGREDIENTS, sin tuplas espurias.

8. ORDERS ⋈ EMPLOYEES

PEDIDOS_EMPLEADO = ORDERS ⋈ EMPLOYEES → hecho que reconstruye: pedidos atendidos por un empleado.

Resultado del JOIN (la columna resaltada es el atributo común)

id_order	order_date	hour	total_price	status	id_employee	name	email	phone	salary	zone
O1	2026-10-01	12:30	43.000	Entregado	E1	Ana	ana@rest.com	3001112233	1.800.000	Terraza
O2	2026-10-01	13:15	54.000	En preparación	E1	Ana	ana@rest.com	3001112233	1.800.000	Terraza
O3	2026-10-02	19:00	50.000	Entregado	E2	Luis	luis@rest.com	3004445566	1.800.000	Salón

Prueba de lossless join

ORDERS ∩ EMPLOYEES = {id_employee}

DF: id_employee → name, email, phone, salary, zone (id_employee es la PK de EMPLOYEES)

{id_employee}⁺ ⊇ EMPLOYEES, por tanto (ORDERS ∩ EMPLOYEES) → EMPLOYEES.

∴ El JOIN es lossless.

Verificación con el ejemplo: ORDERS tiene 3 filas y el resultado tiene 3; cada fila de ORDERS se une con exactamente una fila de EMPLOYEES, sin tuplas espurias.

Marta (E3) no aparece porque no atendió ningún pedido. Eso no es pérdida: no participa en el hecho "pedido atendido por empleado".

9. ORDERDETAILS ⋈ ORDERS

DETALLES_PEDIDO = ORDERDETAILS ⋈ ORDERS → hecho que reconstruye: platos incluidos en un pedido.

Resultado del JOIN (la columna resaltada es el atributo común)

id_order_detail	plate_quantity	id_order	id_plate	order_date	hour	total_price	status	id_employee
OD1	1	O1	P1	2026-10-01	12:30	43.000	Entregado	E1
OD2	1	O1	P2	2026-10-01	12:30	43.000	Entregado	E1
OD3	3	O2	P2	2026-10-01	13:15	54.000	En preparación	E1
OD4	2	O3	P1	2026-10-02	19:00	50.000	Entregado	E2

Prueba de lossless join

ORDERDETAILS ∩ ORDERS = {id_order}

DF: id_order → order_date, hour, total_price, status, id_employee (id_order es la PK de ORDERS)

$\{id_order\}^+ \supseteq$ ORDERS, por tanto (ORDERDETAILS ∩ ORDERS) → ORDERS.

∴ El JOIN es lossless.

Verificación con el ejemplo: ORDERDETAILS tiene 4 filas y el resultado tiene 4; cada fila de ORDERDETAILS se une con exactamente una fila de ORDERS, sin tuplas espurias.

OD1 y OD2 pertenecen al mismo pedido O1 y por eso repiten exactamente la misma fecha, hora y estado: eso es lo que exige la DF id_order → order_date, hour, status.

10. ORDERDETAILS ⋈ PLATES

PLATOS_PEDIDOS = ORDERDETAILS ⋈ PLATES → hecho que reconstruye: qué plato se pidió en cada detalle.

Resultado del JOIN (la columna resaltada es el atributo común)

id_order_detail	plate_quantity	id_order	id_plate	name	price	description
OD1	1	O1	P1	Bandeja paisa	25.000	Plato típico antioqueño
OD2	1	O1	P2	Arroz con pollo	18.000	Arroz con pollo desmechado
OD3	3	O2	P2	Arroz con pollo	18.000	Arroz con pollo desmechado
OD4	2	O3	P1	Bandeja paisa	25.000	Plato típico antioqueño

Prueba de lossless join

ORDERDETAILS ∩ PLATES = {id_plate}

DF: id_plate → name, price, description (id_plate es la PK de PLATES)

$\{id_plate\}^+ \supseteq$ PLATES, por tanto (ORDERDETAILS ∩ PLATES) → PLATES.

∴ El JOIN es lossless.

Verificación con el ejemplo: ORDERDETAILS tiene 4 filas y el resultado tiene 4; cada fila de ORDERDETAILS se une con exactamente una fila de PLATES, sin tuplas espurias.

Control de coherencia: O1 = 25.000 + 18.000 = 43.000; O2 = 3 × 18.000 = 54.000; O3 = 2 × 25.000 = 50.000.

11. PLATE_DETAILS ⋈ PLATES

RECETA_PLATO = PLATE_DETAILS ⋈ PLATES → hecho que reconstruye: componentes (detalles) de cada plato.

Resultado del JOIN (la columna resaltada es el atributo común)

id_detail	quantity	restaurant_price	plate_unit	id_plate	name	price	description
D1	150	1.200	gramos	P1	Bandeja paisa	25.000	Plato típico antioqueño
D2	1	2.000	unidades	P1	Bandeja paisa	25.000	Plato típico antioqueño
D3	200	6.000	gramos	P2	Arroz con pollo	18.000	Arroz con pollo desmechado

Prueba de lossless join

PLATE_DETAILS ∩ PLATES = {id_plate}

DF: id_plate → name, price, description (id_plate es la PK de PLATES)

$\{id_plate\}^+ \supseteq$ PLATES, por tanto (PLATE_DETAILS ∩ PLATES) → PLATES.

∴ El JOIN es lossless.

Verificación con el ejemplo: PLATE_DETAILS tiene 3 filas y el resultado tiene 3; cada fila de PLATE_DETAILS se une con exactamente una fila de PLATES, sin tuplas espurias.

12. INGREDIENTS ⋈ PLATE_DETAILS

INGREDIENTE_EN_PLATO = INGREDIENTS ⋈ PLATE_DETAILS → hecho que reconstruye: en qué detalle de plato se usa cada ingrediente.

Resultado del JOIN (la columna resaltada es el atributo común)

id_ingredient	name	ingredient_unit	minimum_stock	current_stock	id_detail	quantity	restaurant_price	plate_unit	id_plate
I1	Arroz	kilogramos	20	45	D1	150	1.200	gramos	P1
I2	Maduro	unidades	30	60	D2	1	2.000	unidades	P1
I3	Pollo	kilogramos	15	25	D3	200	6.000	gramos	P2

Prueba de lossless join

INGREDIENTS ∩ PLATE_DETAILS = {id_detail}

DF: id_detail → quantity, restaurant_price, plate_unit, id_plate (id_detail es la PK de PLATE_DETAILS)

$\{id_detail\}^+ \supseteq$ PLATE_DETAILS, por tanto (INGREDIENTS ∩ PLATE_DETAILS) → PLATE_DETAILS.

∴ El JOIN es lossless.

Verificación con el ejemplo: INGREDIENTS tiene 3 filas y el resultado tiene 3; cada fila de INGREDIENTS se une con exactamente una fila de PLATE_DETAILS, sin tuplas espurias.

5. Conservación de dependencias

En la tabla de la sección 2, cada DF tiene a la izquierda la PK de una tabla y a la derecha solo atributos de esa misma tabla. Por tanto, cada DF se verifica localmente, y la unión de las DF locales es igual al conjunto F original:

> F_EMPLOYEES ∪ F_SUPPLIERS ∪ F_PURCHASESUPPLIERS ∪ F_PURCHASEDETAILS ∪ F_INGREDIENTS ∪ F_PLATE_DETAILS ∪ F_PLATES ∪ F_ORDERS ∪ F_ORDERDETAILS ∪ F_MOVEMENTS = F
>
> **∴ Se conservan todas las dependencias funcionales.**